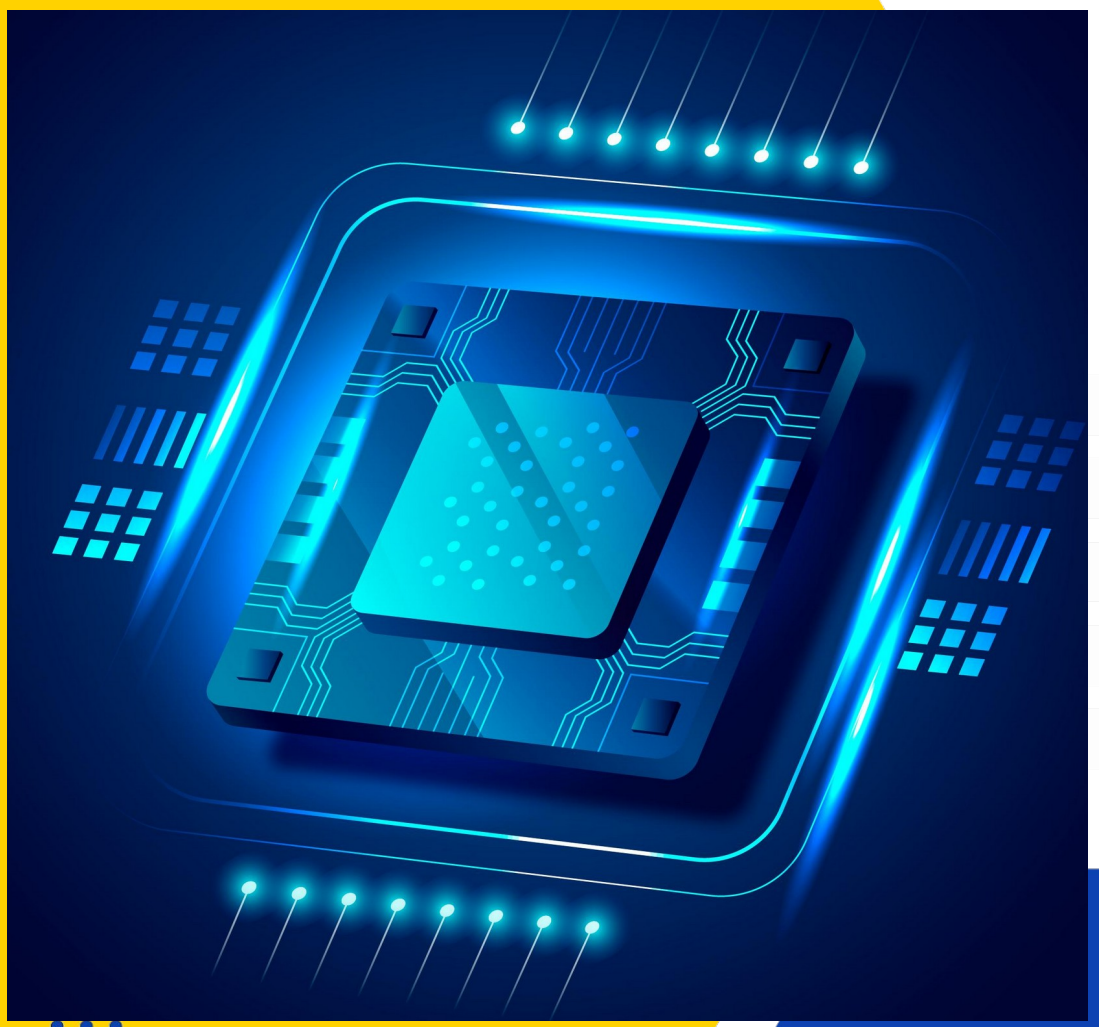


Fonte: Freepik.

Processos e threads em sistemas distribuídos

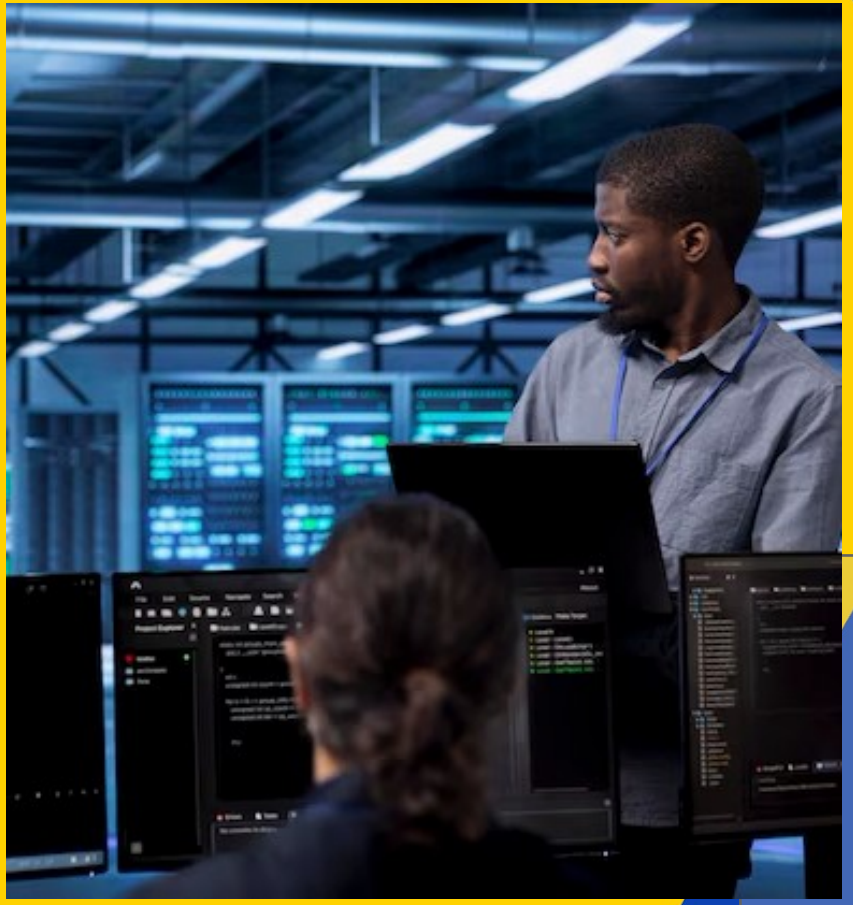
Diogo Augusto Veri

P b ic

Introdução

Vamos entender que os processos e threads desempenham um papel essencial na execução de tarefas, permitindo que sistemas distribuídos operem com eficiência e paralelismo.

Você terá uma visão clara da importância dos processos e threads para o funcionamento desses sistemas complexos.

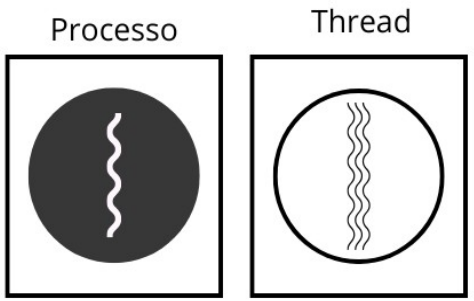


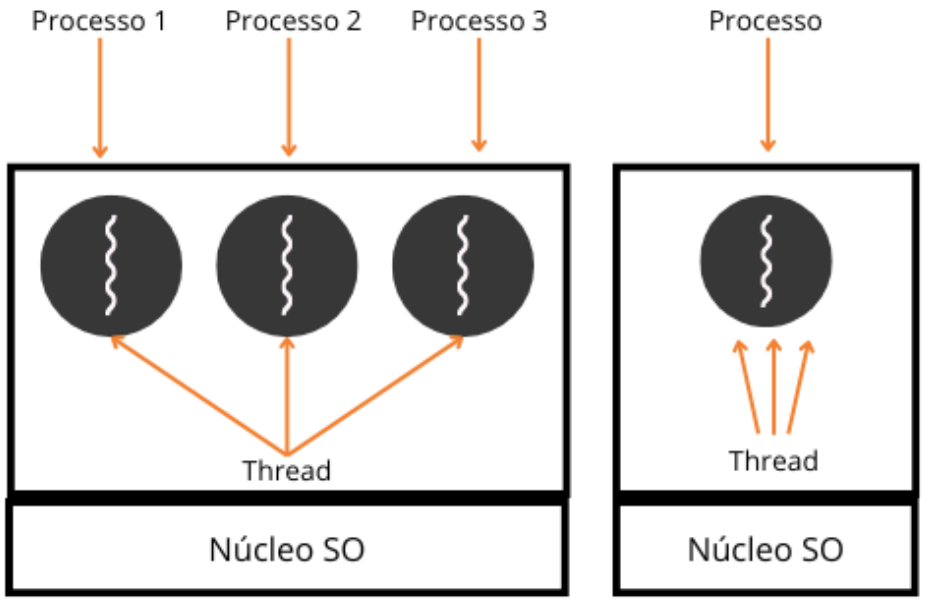
Fonte: Freepik.

P b ic

Introdução aos modelos de processos e threads

- Processos e threads como base dos sistemas distribuídos.
- Diferenças entre processos e threads.
- Importância do paralelismo para eficiência.



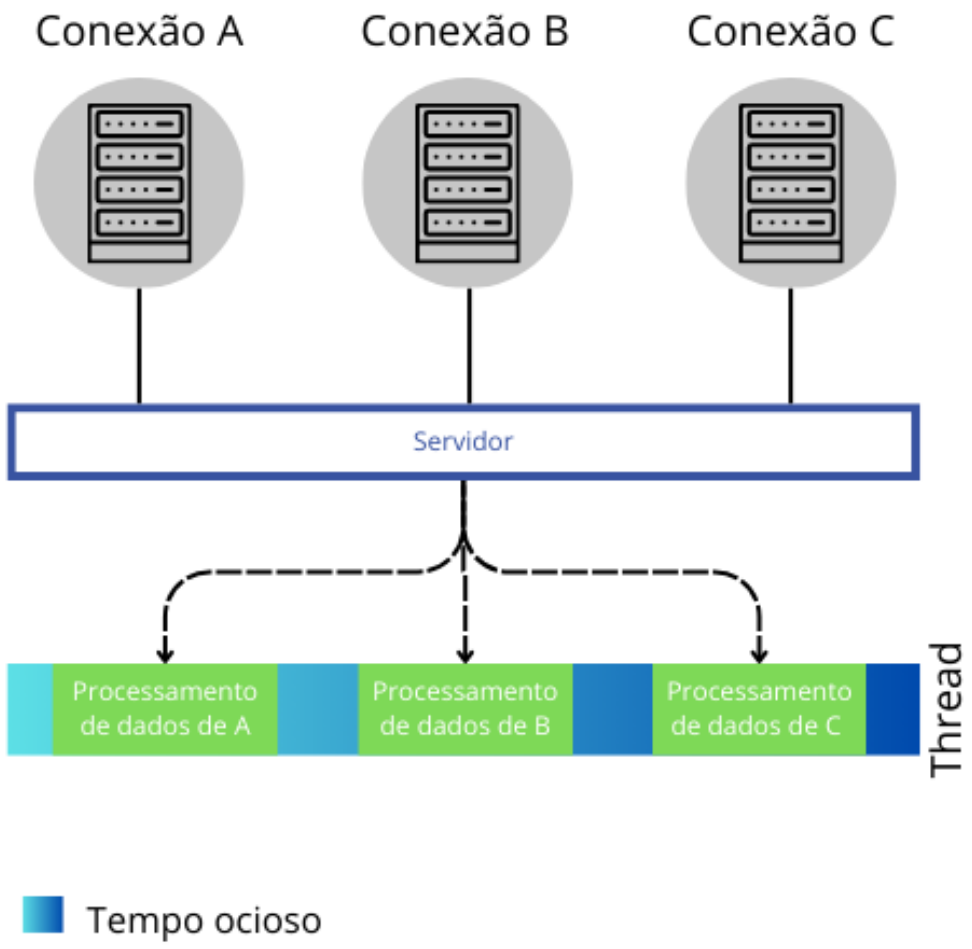


Fonte: elaborada pelo autor.

P b ic

Modelo de processos em sistemas distribuídos

- Processos independentes.
- Separação de memória e recursos.
- Escalabilidade em diferentes máquinas.

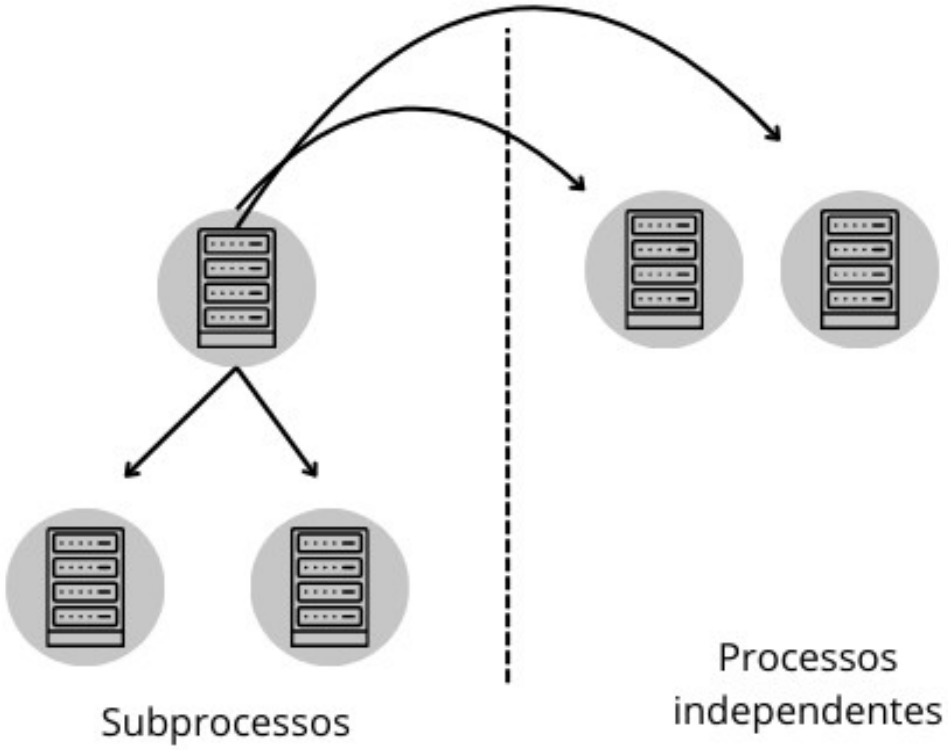


Fonte: elaborada pelo autor.

P b ico

Modelo de threads em sistemas distribuídos

- Threads como subprocessos leves.
- Compartilhamento de recursos e memória.
- Redução de consumo de recursos.



Fonte: elaborada pelo autor.

P ̇blico

Criação e gerenciamento de processos

- Papel do sistema operacional.
- Criação e finalização de processos.
- Distribuição de tarefas entre máquinas.

Fonte: elaborada pelo autor.

P b ico

Criação e gerenciamento de threads

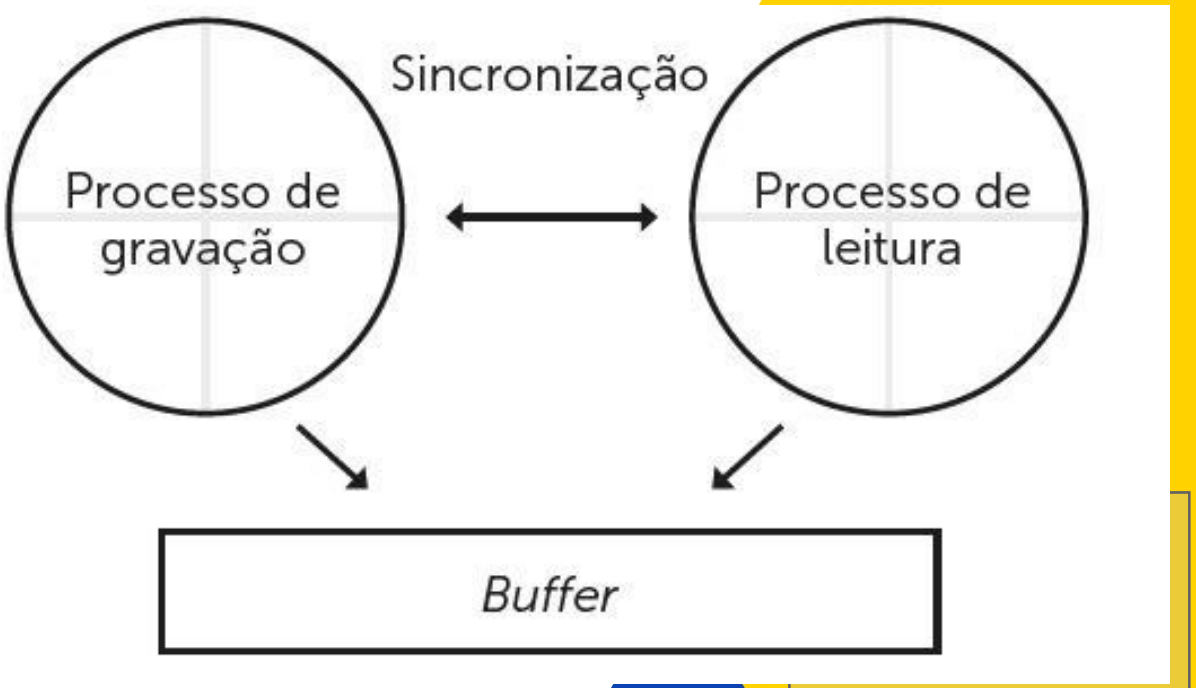
- Threads versus processos.
- Gerenciamento eficiente em paralelismo.
- Evitar gargalos no sistema.

Fonte: Freepik.

P b ico

Comunicação interprocessos (IPC)

- Troca de mensagens entre processos.
- Uso de buffers compartilhados.
- Importância da sincronização.



Fonte: elaborada pelo autor.

P b ico

Comunicação entre threads

- Sincronização de threads.
- Uso de semáforos, mutexes e barreiras.
- Evitando conflitos em acesso concorrente.

Fonte:

P b ico

Exemplos práticos de processos e threads

- Sistemas bancários: gerenciamento de transações.
- E-commerce: processamento de pedidos em paralelo.
- Streaming: execução simultânea de múltiplos fluxos de dados.

Fonte: Freepik.

P ̇blico

Conclusão e importância dos processos e threads

- Processos e threads em sistemas distribuídos.
- Garantia de escalabilidade e desempenho.
- Necessidade de sincronização para evitar conflitos.

Fonte: Freepik.

Público

Bons estudos!

Fonte: Freepik.

Public